

Programowanie współbieżne w C++

Lista zadań nr 14 (bonusowa)

termin zgłoszenia: 28 czerwca 2026

Tematy różne

Zadanie 1 (6pkt). Mamy zasób, na którym działają 3 typy wątków: przeszukujące, wstawiające i usuwające. Współbieżne dostępy do tego zasobu muszą przestrzegać następujących zasad: a) jednocześnie może działać wiele wątków przeszukujących, b) jednocześnie może działać jeden wątek wstawiający, c) wątki przeszukujące mogą działać jednocześnie z wątkiem wstawiającym, d) wątek usuwający nie może pracować współbieżnie z żadnymi innymi wątkami, w tym z innymi wątkami usuwającymi. Zaimplementuj klasę **ResourceManager** z metodami:

```
void start_search()    // Wejście wątków przeszukujących
void end_search()      // Wyjście wątek przeszukujących

void start_insert()    // Wejście wątku wstawiającego
void end_insert()      // Wyjście wątku wstawiającego

void start_delete()    // Wejście wątku usuwającego
void end_delete()      // Wyjście wątku usuwającego
```

zachowującymi podaną semantykę. Można używać blokad. Przetestuj swoje rozwiązanie!

Zadanie 2 (5+1pkt). Zaimplementuj klasę **ConcurrentTaskEngine**¹, która będzie silnikiem obliczeniowym umożliwiającym równoległe wykonywanie wielu zadań obliczeniowych. Zadania będą dynamicznie dodawane do kolejki zadań, a silnik będzie zarządzał ich współbieżnym wykonaniem oraz zwracaniem wyników. Jako kolejkę zadań wykorzystaj strukturę danych z blokadami.

Wymagania szczegółowe

1. Klasa **ConcurrentTaskEngine**:

- utrzymuje wewnętrzną kolejkę zadań do wykonania,
- pozwala na dynamiczne dodawanie nowych zadań w trakcie działania programu,

¹ Bardzo podobne zadanie było już na liście 3.

- zarządza pulą wątków (o rozmiarze podanym w konstruktorze), które pobierają zadania z kolejki i wykonują je współbieżnie,
- zwraca wyniki wykonanych zadań za pomocą mechanizmu [`std::future`](#),
- jest gotowa do przyjmowania nowych zadań, aż do momentu wywołania funkcji `shutdown`.

Silnik powinien obsługiwać dowolne zadania obliczeniowe (np. obliczenia matematyczne). Każde zadanie powinno być opakowane w [`std::packaged\_task`](#), a wynik zwracany użytkownikowi poprzez `std::future`. Silnik powinien umożliwiać bezpieczne i eleganckie zakończenie pracy (graceful shutdown).

Oto przykładowy interfejs klasy. Możesz zmienić nagłówek funkcji `submit`, ale utracisz wtedy punkt bonusowy.

```
class ConcurrentTaskEngine {
public:
    explicit ConcurrentTaskEngine(size_t thread_count);
    ~ConcurrentTaskEngine();

    // Dodaje nowe zadanie do kolejki i zwraca std::future na wynik
    template <typename Func, typename... Args>
    auto submit(Func&& func, Args&&... args) ->
    std::future<decltype(func(args...))>; // Może przydać się std::function.

    // Bezpiecznie kończy działanie silnika (czeka na zakończenie
    // wszystkich zadań)
    void shutdown();

private:
    // Tu umieść metody prywatne do zarządzania pulą wątków i kolejką
    // zadań.
};
```

Przykładowe użycie klasy:

```
ConcurrentTaskEngine engine(4); // 4-wątkowy silnik

auto future1 = engine.submit([](int x) {
    std::this_thread::sleep_for(std::chrono::seconds(2));
    return x * x;
}, 10);

auto future2 = engine.submit([]() {
    std::this_thread::sleep_for(std::chrono::seconds(1));
    return std::string("Hello from task!");
});

try {
    int result1 = future1.get(); // Oczekiwanie na wynik pierwszego zadania
    std::string result2 = future2.get(); // Oczekiwanie na wynik drugiego
    // zadania

    std::cout << "Wynik zadania 1: " << result1 << "\n";
```

```

        std::cout << "Wynik zadania 2: " << result2 << "\n";
    } catch (const std::exception& e) {
        std::cerr << "Wyjątek podczas wykonywania zadania: " << e.what() << "\n";
    }

    engine.shutdown();

```

Przetestuj swoją implementację!

Zadanie 3 (3+1pkt). Zastanów się, czy klasa opracowana w poprzednim zadaniu nadaje się do implementacji wielowątkowego sortowania przez scalanie. Zaimplementuj ten algorytm z użyciem tej klasy lub jej minimalnej modyfikacji. Oszczędzaj pamięć! Bonusowy punkt otrzymasz za generyczny algorytm sortowania, działający dla tablic elementów typu `T` z operatorem `<`.

Zadanie 4 (5+3pkt) ConcurrentTaskEngine z podkradaniem zadań i blokadami.

Wykonaj reimplementację klasy `ConcurrentTaskEngine` (z zadania 2), w której każdy wątek z puli będzie pobierał zadania z własnej kolejki zadań. Funkcja `submit` powinna balansować obciążenie wątków rozdzielając zadania uczciwie (np. metodą round-robin). Ponadto, wątek który opróżnił swoją kolejkę zadań powinien spróbować wykraść zadanie innemu, losowo wybranemu wątkowi. Trudność zadania polega na zsynchronizowaniu dostępu do kolejki zadań przynależącej do wątku: dostępu do niej potrzebuje wątek właściciel, wątek (wątki?) wykonujący `submit` oraz wątki wykonujące funkcję podkradania. Punkty bonusowe otrzymasz, jeśli zastosujesz bardziej drobnoziarnistą niż pojedynczy mutex blokadę kolejki zadań. Kolejkę zadań możesz zaimplementować w dowolny sposób.

Zadanie 5 (5pkt) .Współbieżna kolejka na dwóch okresowo zsynchronizowanych stosach.

Zaimplementuj szablon klasy `ThreadSafeTwoStackQueue<T>` realizujący kolejkę FIFO ograniczonego rozmiaru z użyciem dwóch wewnętrznych stosów (`std::vector<T>`). Struktura musi zapewniać bezpieczeństwo wątkowe poprzez separację mechanizmów blokujących dla operacji dodawania (na stos `enqueue_stack`) i usuwania (ze stosu `dequeue_stack`). Zdefiniuj blokujące metody `push` i `pop` oraz ich nieblokujące odpowiedniki `try_push` i `try_pop`, które zgłaszają błąd przy próbie operacji na przepełnionej/opróźnionej kolejce. W przypadku wykrycia

pustego `dequeue_stack`, automatycznie przetransferuj elementy z `enqueue_stack` z zachowaniem kolejności. Zidentyfikuj wszystkie miejsca potencjalnych wyjątków (w tym operacje na pamięci i konstruktory przenoszące typu `T`), zapewniając spójność struktury danych w przypadku ich wystąpienia. Zadbaj o minimalizację czasu blokady poprzez użycie oddzielnych muteksów dla każdego stosu i podwójnego blokowania podczas transferu między stosami.

Zadanie 6 (7pkt). Współbieżna leniwa lista z przeskokami.

Zapoznaj się z algorytmem współbieżnej leniwej listy z przeskokami (klasa `LazySkipList` z rozdziału 14.3 książki TAoMP2e), następnie zaimplementuj ją w C++ i dobrze przetestuj. Zignoruj problem zwalniania pamięci.

Zadanie 7 (7pkt). Współbieżna kolejka priorytetowa.

Zapoznaj się z algorytmem współbieżnej kolejki priorytetowej bazującej na liście z przeskokami (klasa `PrioritySkipList` z rozdziału 15.5 TAoMP2e), następnie zaimplementuj ją w C++ i dobrze przetestuj. Podobnie jak w poprzednich zadaniach, zadbaj o odpowiedni interfejs. Tutaj również możesz zignorować problem zwalniania pamięci.

Zadanie 8 (2pkt). Równoległe obliczanie sum częściowych metodą podziału problemu

Uruchom i **przetestuj** algorytm dla problemu obliczania sum częściowych metodą podziału problemu z rozdziału 8.5.3 książki C++ Concurrency in Action, 2e. Upewnij się, że rozumiesz jak działa zaimplementowany algorytm. W jakich warunkach ten algorytm ma szansę działać efektywnie? W tym zadaniu oceniane będzie dostarczenie masywnych testów algorytmu oraz pokrycie przypadków brzegowych.

Zadanie 9 (3pkt). Równoległe obliczanie sum częściowych z użyciem barier.

Powtórz poprzednie zadanie dla algorytmu używającego barier (również znajdującego się w rozdziale 8.6.3 książki wymienionej w poprzednim zadaniu). Tutaj też oceniania będzie jakość testów.